

ManelCore

Plateforme IA de Développement des Affaires

Documentation Technique Complète

Version 1.0 — Mai 2025

Manel Canada | zebazemadric@icloud.com

Python 3.14

FastAPI

LangGraph

Neo4j

Flutter

LM Studio

Table des matières

1.	Vue d'ensemble du projet	3
2.	Architecture système	4
3.	Backend — FastAPI & Python	5
4.	Agents IA — LangGraph	7
5.	Base de données — Neo4j Graph	10
6.	Services métier	12
7.	Frontend — Flutter	15
8.	API REST — Référence des endpoints	17
9.	Infrastructure & déploiement	19
10.	Flux de données end-to-end	20
11.	Configuration & variables d'environnement	21
12.	Sécurité & bonnes pratiques	22
13.	Roadmap & évolutions	23

1. Vue d'ensemble du projet

ManelCore est une plateforme intelligente de développement des affaires conçue pour automatiser la veille d'opportunités (appels d'offres, emplois), la gestion des contacts, la rédaction et l'envoi de courriels professionnels, ainsi que la gestion des candidatures RH — le tout piloté par des agents IA fonctionnant entièrement en local grâce à LM Studio.

Objectifs stratégiques

- **Veille automatisée** : scanner SEAO (appels d'offres québécois), LinkedIn Jobs et Indeed pour identifier en continu les opportunités pertinentes.
- **Scoring IA** : classer chaque opportunité par pertinence métier grâce à un LLM local (Gemma 4 via LM Studio).
- **Outreach automatisé** : générer des courriels de prospection personnalisés et les soumettre à validation humaine avant envoi.
- **Graph intelligence** : stocker toutes les entités (entreprises, contacts, opportunités, messages) dans Neo4j pour exploiter les relations.
- **Interface unifiée** : une application Flutter cross-platform (macOS, Linux, iOS, Android) comme tableau de bord centralisé.
- **IA conversationnelle** : un chatbot ARIA (Graph RAG) pour interroger les données de l'entreprise en langage naturel.
- **Intégrations** : Telegram bot pour les notifications push, agent email IMAP pour la réponse automatique.

Principes de conception

Principe	Application concrète
Local-first	LM Studio — aucune donnée ne quitte l'infrastructure
Human-in-the-loop	Validation obligatoire avant envoi de courriel ou acceptation d'opportunité
Graph-native	Neo4j comme source de vérité relationnelle plutôt qu'une BDD relationnelle plate
Streaming SSE	Retour temps réel de l'avancement des agents vers le frontend
Modulaire	Agents LangGraph indépendants composables selon le flux métier

2. Architecture système

ManelCore suit une architecture **client-serveur découplée** avec un backend Python asynchrone, une base de données graphe et un frontend Flutter. Les agents IA s'exécutent comme des processus async dans le même processus uvicorn.

Diagramme des couches

Couche	Technologie	Rôle
Présentation	Flutter + Riverpod	UI cross-platform, état réactif, SSE consumer
API Gateway	FastAPI (uvicorn)	Exposition REST + SSE, orchestration des agents
Agents IA	LangGraph + LangChain	Graphes d'état pour Explorer et Contact agents
LLM Runtime	LM Studio (Gemma 4)	Inférence locale OpenAI-compatible sur :1234
Persistance	Neo4j Desktop	Graphe de connaissances sur bolt://localhost:7687
Messaging	SMTP/IMAP + Telegram	Email sortant/entrant et notifications push
RAG	RagService (Neo4j)	Récupération contextuelle avant chaque appel LLM chat

Flux de communication inter-composants

Le frontend Flutter communique exclusivement via HTTP/SSE avec le backend FastAPI sur le port **8000**. Les agents LangGraph s'exécutent dans le même processus et communiquent avec Neo4j via Bolt (**7687**) et avec le LLM via l'API OpenAI-compatible de LM Studio sur le port **1234**. Les services email (IMAP/SMTP) et Telegram fonctionnent en tâches asyncio de fond.

Port	Service	Protocole	Description
8000	FastAPI Backend	HTTP / SSE	API principale + streaming temps réel
7687	Neo4j Desktop	Bolt	Base de données graphe
1234	LM Studio	HTTP OpenAI API	Inférence LLM locale (Gemma 4)
IMAP/SMTP	Serveur mail	TLS	Lecture/envoi des courriels

Telegram	Bot API	HTTPS Webhook	Notifications et commandes
----------	---------	---------------	----------------------------

3. Backend — FastAPI & Python

Structure des fichiers

```

backend/
■■■ main.py # Point d'entrée uvicorn
■■■ .env # Variables d'environnement (secrets)
■■■ requirements.txt
■■■ app/
■■■ api/
■ ■■■ main.py # Tous les endpoints FastAPI
■■■ agents/
■ ■■■ contact/ # Agent de rédaction email
■ ■ ■■■ graph.py
■ ■ ■■■ nodes.py
■ ■ ■■■ state.py
■ ■■■ explorer/ # Agent de veille d'opportunités
■ ■■■ graph.py
■ ■■■ nodes.py
■ ■■■ state.py
■■■ database/
■ ■■■ connection.py # Driver Neo4j
■ ■■■ models.py # Dataclasses (Entreprise, Contact...)
■ ■■■ queries.py # Contraintes Cypher
■ ■■■ repository.py # CRUD Neo4j
■ ■■■ schema.py # Initialisation et seed
■■■ services/
■■■ email_agent.py # Agent IMAP + réponse auto
■■■ mailer.py # SMTP + IMAP client
■■■ opportunity_crawler.py # LinkedIn + Indeed
■■■ rag_service.py # Graph RAG pour le chat
■■■ seao_api.py # API publique SEO
■■■ settings_service.py # Persistance JSON des settings
■■■ telegram_service.py # Bot Telegram polling

```

Démarrage du serveur

```

# Depuis la racine du projet
cd backend
python main.py
# ou directement :
uvicorn app.api.main:app --reload --port 8000

```

Cycle de démarrage FastAPI

Au démarrage (@app.on_event('startup')), le backend initialise deux tâches asyncio de fond :

- **Bot Telegram** : démarre le polling si TELEGRAM_BOT_TOKEN est configuré.
- **Email agent** : lance une boucle de polling IMAP toutes les 5 minutes pour lire et traiter les courriels entrants.

Dépendances principales

Package	Rôle
fastapi	Framework web asynchrone
uvicorn[standard]	Serveur ASGI avec support WebSocket
langgraph	Orchestration des agents sous forme de graphes d'état
langchain-core / langchain-openai	Abstractions LLM et intégration OpenAI-compatible
neo4j	Driver officiel Neo4j (Bolt)
imap-tools	Lecture et gestion des courriels IMAP
python-telegram-bot	Client Telegram Bot API
python-dotenv	Chargement des variables d'environnement
pydantic	Validation des données et modèles Pydantic
httpx	Client HTTP async pour les APIs externes
sse-starlette	Server-Sent Events pour le streaming frontend

4. Agents IA — LangGraph

ManelCore utilise **LangGraph** pour orchestrer deux agents IA distincts, chacun modélisé comme un graphe d'état dirigé (*StateGraph*). Chaque nœud du graphe est une fonction asynchrone pure qui lit et écrit dans un état partagé typé (*TypedDict*). Le LLM utilisé est **Google Gemma 4 (e4b)** via LM Studio sur localhost:1234, exposé avec une interface compatible OpenAI.

4.1 Agent Explorer — Veille d'opportunités

L'Agent Explorer effectue une recherche multi-sources, analyse les résultats avec le LLM et les persiste dans Neo4j. Il supporte les interruptions pour révision humaine.

Graphe d'exécution

Nœud	Fonction	Description
load_profile	load_profile()	Charge le profil entreprise + secteurs depuis Neo4j
generate_queries	generate_queries()	Génère 3 requêtes ciblées via LLM (ou fallback)
search_seao	search_seao()	Interroge l'API publique SEO (appels d'offres QC)
search_linkedin	search_linkedin()	Extrait les offres LinkedIn via jobs-guest API
search_indeed	search_indeed()	Scrape les flux RSS Indeed Canada
rank_and_analyze	rank_and_analyze()	Déduplique, score par pertinence, persiste en Neo4j
human_review	human_review()	Point d'interruption — attente validation utilisateur

Flux d'exécution parallèle

Après `generate_queries`, le graphe effectue un **fan-out parallèle** vers les trois sources (SEAO, LinkedIn, Indeed) simultanément. Les résultats fusionnent dans `rank_and_analyze` grâce à l'opérateur `operator.add` sur la liste `found_opportunities` de l'état.

État Explorer (ExplorerState)

```
class ExplorerState(TypedDict):
    company_profile: str
    sectors: List[str]
    search_queries: List[str]
    found_opportunities: Annotated[List[dict], operator.add] # fan-in
    ranked_opportunities: List[dict]
```



```
messages: Annotated[List[BaseMessage], operator.add]
current_source: Annotated[List[str], operator.add]
approved_opportunities: List[dict]
review_comment: str
errors: Annotated[List[str], operator.add]
```

Persistence (Neo4j) dans rank_and_analyze

Pour chaque opportunité classée, le nœud appelle repo.upsert_opportunite() avec les champs titre, source, URL, statut, score de pertinence, organisation et dates. En cas de doublon (même URL ou titre), Neo4j effectue un MERGE pour éviter les duplicatas. L'analyse LLM ajoute deux champs clés : score_pertinence (0.0–1.0) et resume (analyse critique 4-6 phrases).

4.2 Agent Contact — Rédaction et envoi d'emails

L'Agent Contact orchestre la rédaction d'un courriel de prospection personnalisé et son envoi via SMTP après validation humaine explicite. Il utilise une interruption LangGraph (interrupt_before=["send_email"]) pour garantir le contrôle humain.

Nœuds du graphe

Nœud	Action
fetch_history	Charge l'opportunité cible + profil entreprise + historique des échanges Neo4j
draft_response	Génère un courriel professionnel via LLM avec prompt structuré riche
send_email	Envoie via SMTP et persiste le message dans Neo4j (si approved=True)

Prompt de rédaction — Structure

```
Tu es l'assistant de développement des affaires de {company_name}.

=== NOTRE ENTREPRISE ===
{company_context}

=== OPPORTUNITÉ CIBLÉE ===
{opp_context} # titre, organisation, type, budget, résumé, exigences

=== CONTACT ===
Nom / Organisation / Poste

=== HISTORIQUE DES ÉCHANGES ===
{history_text} # derniers 50 messages liés à l'opportunité

CONSIGNES: accroche personnalisée, 2-3 points forts, CTA concret,
max 250 mots, ton chaleureux et professionnel.

FORMAT: Objet: [sujet accrocheur]\n\n[corps du courriel]
```

État Contact (ContactState)

```
class ContactState(TypedDict):
    opportunity_id: str
    contact_info: dict # email, nom, organisation, poste
    opportunity_details: dict # données Neo4j complètes
    company_context: dict # Entreprise + ProfilEntreprise
    conversation_history: List[dict]
    draft_email: str
    status: str # drafting|awaiting_approval|approved|sent|error
    approved: bool
    error: str
    messages: Annotated[List[BaseMessage], operator.add]
```

4.3 LLM Runtime — LM Studio

Les deux agents utilisent ChatOpenAI de LangChain pointant sur LM Studio. Le modèle par défaut est **google/gemma-4-e4b**, configurable via la variable MODEL. Un timeout configurable (SEARCH_LLM_TIMEOUT_SECONDS, défaut 30s) protège contre les inférences trop longues.

Modèle	google/gemma-4-e4b (configurable via MODEL env)
Base URL	http://localhost:1234/v1 (configurable via MODEL_BASE_URL)
API Key	lm-studio (fictif, requis par le client OpenAI)
Max tokens	1500 (Contact) / 2000 (Explorer)
Timeout	30 secondes par défaut (SEARCH_LLM_TIMEOUT_SECONDS)

5. Base de données — Neo4j Graph

ManelCore utilise **Neo4j Desktop** comme base de données principale. Le choix d'une base de données graphe est fondamental : les entités métier (entreprises, contacts, opportunités, messages) sont naturellement interconnectées, et Neo4j permet d'exploiter ces relations avec le langage **Cypher** de façon beaucoup plus expressive qu'un SQL avec jointures multiples.

5.1 Modèle de données — Nœuds

Label	Champs clés	Description
Entreprise	nom, site_web, description, ville, pays, secteur_principal	L'entreprise utilisatrice (Manel Canada)
ProfilEntreprise	resume, points_forts, faiblesses, services, historique	Analyse stratégique de l'entreprise pour le RAG
Contact	nom, email, telephone, poste, linkedin, source	Contacts professionnels (clients, prospects)
Opportunite	titre, type, source, url, statut, date_limite, score_pertinence, resume	Appels d'offres, contrats, emplois
Secteur	nom, description	Domaines d'activité cibles
Message	canal, sujet, contenu, direction, date_envoi, from_email, intent, sentiment	Emails, messages Telegram, interactions
Conversation	canal, statut, sujet, created_at	Fil de discussion groupant des Messages
Document	nom, type, url, contenu_extrait, embedding_id	Fichiers joints, propositions, CV
Candidature	statut, date_soumission, proposition, montant	Réponses aux appels d'offres
AgentAction	type, statut, input, output, erreur	Traçabilité des actions des agents IA
Besoin	nom, description, priorite	Besoins exprimés par l'entreprise

5.2 Relations (Edges)

Relation	De → Vers	Signification
TRAVAILLE_DANS	Entreprise → Secteur	L'entreprise opère dans ce secteur
PUBLIE	Entreprise → Opportunite	L'entreprise a publié l'appel d'offres

APPARTIENT_A	Opportunité → Secteur	L'opportunité appartient à ce secteur
A_CONTACT	Opportunité → Contact	Personne de contact pour l'opportunité
A_CONVERSATION	Opportunité → Conversation	Fil d'échanges lié à l'opportunité
CONTIENT	Conversation → Message	Message inclus dans la conversation
A_SOUMIS	Entreprise → Candidature	Proposition soumise pour l'opportunité
CONCERNANT	Candidature → Opportunité	Candidature associée à l'opportunité
A_DOCUMENT	Candidature → Document	Pièce jointe de la candidature
A_EFFECTUE	AgentAction → *	Action d'agent sur une entité

5.3 Connexion et Repository

La classe `Neo4jConnection` encapsule le driver officiel Python de Neo4j. Elle supporte l'utilisation comme context manager (with `Neo4jConnection()` as `conn`) pour garantir la fermeture des sessions. La configuration est chargée en priorité depuis les variables d'environnement, puis depuis le fichier JSON des settings persistés.

Méthodes principales du GraphRepository

Méthode	Description
<code>upsert_entreprise()</code>	Crée ou met à jour l'entité Entreprise (MERGE par nom)
<code>upsert_opportunite()</code>	Crée ou met à jour une opportunité (MERGE par titre+source)
<code>upsert_contact()</code>	Crée ou met à jour un contact (MERGE par email)
<code>upsert_message()</code>	Persiste un message avec métadonnées complètes
<code>upsert_secteur()</code>	Crée ou met à jour un secteur d'activité
<code>get_node(label, id)</code>	Récupère un nœud par son ID
<code>find_nodes(label, ...)</code>	Recherche des nœuds avec filtre, tri et pagination
<code>get_related_nodes()</code>	Traverse les relations pour trouver les nœuds voisins
<code>create_relationship()</code>	Crée une relation orientée entre deux nœuds
<code>delete_node(label, id)</code>	Supprime un nœud et ses relations
<code>count_nodes(label)</code>	Retourne le nombre de nœuds d'un label donné
<code>get_dashboard_stats()</code>	Agrégation pour le tableau de bord (comptages globaux)

6. Services métier

6.1 MailerService — SMTP & IMAP

Le MailerService gère l'envoi de courriels via SMTP avec TLS et la récupération des messages non lus via IMAP. Il supporte les pièces jointes et les messages HTML/texte. Trois comptes email distincts sont configurables : prospection, RH et entreprise général.

Envoi SMTP	smtplib avec STARTTLS, support HTML + texte alternatif
Lecture IMAP	imap-tools, récupère les 20 derniers non-lus avec métadonnées complètes
Comptes	MAILER_EMAIL (prospection), MAILER_HR_EMAIL (RH), MAILER_COMPANY_EMAIL (général)
Pièces jointes	Métadonnées collectées (nom, type MIME, taille) sans lire le contenu binaire

6.2 EmailAgent — Réponse automatique IMAP

L'EmailAgent s'exécute en boucle de fond toutes les **5 minutes**. Il récupère les courriels non lus de *tous* les comptes configurés, les analyse avec le LLM pour en extraire l'intention et le sentiment, et peut générer des réponses automatiques pour les candidatures RH (compte MAILER_HR). Chaque courriel traité est persisté dans Neo4j avec une classification automatique.

6.3 RagService — Graph RAG pour le Chat

Le RagService construit dynamiquement un bloc de contexte système injecté avant chaque message LLM du chat (ARIA). Pour chaque requête utilisateur, il :

- Extrait les mots-clés significatifs de la question (filtre stopwords FR/EN, longueur > 3).
- Charge le profil entreprise + ProfilEntreprise depuis Neo4j.
- Score les opportunités par overlap de mots-clés avec la requête.
- Ajoute les contacts récents, statistiques emails et candidatures actives.
- Détecte si la question concerne les emails (mots-clés : 'mail', 'courriel', etc.) et charge la boîte de réception.
- Retourne un contexte formaté Markdown injecté comme message système.

6.4 SeaoApiService — Appels d'offres québécois

Le SeaoApiService interroge l'**API publique du SEAO** (Système électronique d'appel d'offres du Québec) via HTTPS. Pour chaque requête de recherche, il normalise les résultats et construit l'URL de l'avis publié sur seao.gouv.qc.ca. L'endpoint utilisé est api.seao.gouv.qc.ca/prod/api/recherche.

6.5 OpportunityCrawlerService — LinkedIn & Indeed

Ce service interroge deux sources :

- • **LinkedIn Jobs** : API jobs-guest non authentifiée — extraction des titres, organisations, URLs et descriptions depuis la réponse JSON publique.
- • **Indeed Canada** : flux RSS ca.indeed.com/rss — parsing XML pour extraire titre, lien, organisation et date.

Les résultats sont normalisés dans un format unifié avant d'être passés à rank_and_analyze. Un champ source ('LinkedIn' ou 'Indeed') est toujours préservé pour la traçabilité.

6.6 TelegramService — Notifications push

Le TelegramService démarre un bot Telegram en mode polling au lancement de l'application (si TELEGRAM_BOT_TOKEN est défini). Il permet de :

- • Recevoir des commandes Telegram (ex: /status, /run) pour déclencher des actions depuis le mobile.
- • Envoyer des notifications push lors d'événements importants (nouvelles opportunités, courriels reçus).
- • Interagir avec l'entreprise de manière asynchrone sans ouvrir l'application Flutter.

6.7 SettingsService — Configuration persistante

Les paramètres modifiables par l'utilisateur (tokens API, credentials email, paramètres Neo4j, profil entreprise) sont stockés dans un fichier JSON (settings.json) dans le répertoire de données de l'application. Ce fichier est chargé au démarrage et fusionné avec les variables d'environnement (les variables d'env ont la priorité).

7. Frontend — Flutter

Le frontend ManelCore est une application **Flutter** cross-platform (macOS, Linux, iOS, Android) utilisant **Riverpod** pour la gestion d'état réactive. L'architecture suit le pattern **Feature-first** : chaque fonctionnalité est un répertoire autonome sous lib/features/.

7.1 Structure de navigation

Index	Page	Route API	Description
0	Dashboard	/dashboard/stats	KPIs globaux, lancement du cycle de veille
1	Opportunités	/opportunities	Liste filtrée + CRUD des opportunités
2	Recherche	/agent/*	Lancement cycle + streaming SSE temps réel
3	Validations	/opportunities?statut=en_attente	Approbation/rejet d'opportunités
4	Contacts	/contacts	Carnet d'adresses + rédaction email via Contact Agent
5	Chat (ARIA)	/chat/stream	Chatbot IA avec Graph RAG + streaming
6	Boîte mail	/email/*	Lecture IMAP + envoi SMTP
7	RH	/candidats	Gestion des candidatures et candidats
8	Planificateur	—	Planning et agenda (à implémenter)
9	Paramètres	/config, /settings	Configuration complète de l'application

7.2 ApiClient & Riverpod Providers

Toute communication avec le backend passe par la classe ApiClient (lib/core/api_client.dart). Les données sont exposées via des FutureProvider et FutureProvider.family Riverpod, permettant un rafraîchissement déclaratif avec ref.invalidate().

Provider	Type	Source
apiClientProvider	Provider	Singleton ApiClient

dashboardStatsProvider	FutureProvider	GET /dashboard/stats
opportunitiesProvider	FutureProvider.family	GET /opportunities?statut=...
contactsProvider	FutureProvider	GET /contacts
candidatsProvider	FutureProvider	GET /candidats
agentStatusProvider	FutureProvider	GET /agent/status
configProvider	FutureProvider	GET /config

7.3 Streaming SSE — Agent Explorer

La page Recherche utilise `ApiClient.streamAgentEvents()` pour consommer un flux SSE (GET /agent/stream). Chaque événement JSON reçu est affiché en temps réel : source interrogée, URL visitée, nombre d'opportunités trouvées. Un second flux (/agent/live-stream) diffuse les étapes détaillées de navigation avec source, action et timestamp.

7.4 Chat ARIA — Streaming token par token

La page Chat utilise `ApiClient.chatStream()` qui ouvre une connexion HTTP persistante (POST /chat/stream) et yield les tokens de réponse LLM au fur et à mesure. Le premier événement peut contenir les métadonnées RAG (`{'rag': {...}}`) pour indiquer quelles données Neo4j ont été injectées dans le contexte.

7.5 Floating Action Button — Cycle rapide

Le FAB animé (`_CycleFab`) est visible depuis toutes les pages (sauf Recherche). Il propose deux actions rapides :

- **Cycle de test** : injecte 5 opportunités fictives via POST /agent/run/mock pour tester l'interface sans lancer le cycle complet.
- **Cycle complet** : navigue vers la page Recherche pour lancer le cycle Explorer complet avec streaming.

8. API REST — Référence des endpoints

Méthode	Endpoint	Description
GET	/health	Health check — état Neo4j + LLM
GET	/dashboard/stats	Comptages globaux pour le tableau de bord
GET	/config	Profil entreprise (Entreprise + ProfilEntreprise)
POST	/config	Met à jour le profil entreprise dans Neo4j
GET	/settings	Paramètres JSON persistés (tokens, credentials)
POST	/settings	Sauvegarde les paramètres JSON
DELETE	/data/erase	Efface toutes les données Neo4j (DANGER)
GET	/opportunities	Liste des opportunités (filtre statut, limite)
POST	/opportunities	Crée une opportunité manuellement
PUT	/opportunities/{id}	Met à jour une opportunité
PATCH	/opportunities/{id}/status	Change le statut d'une opportunité
DELETE	/opportunities/{id}	Supprime une opportunité
GET	/contacts	Liste tous les contacts
POST	/contacts	Crée un contact
PUT	/contacts/{id}	Met à jour un contact
DELETE	/contacts/{id}	Supprime un contact
POST	/agent/run	Démarre le cycle Explorer (async, non bloquant)
POST	/agent/run/mock	Injecte 5 opportunités fictives de test
GET	/agent/status	État courant de l'agent Explorer
GET	/agent/stream	SSE — événements temps réel de l'agent
GET	/agent/live-stream	SSE — étapes de navigation détaillées

POST	/contact/draft	Lance le Contact Agent (génère brouillon email)
POST	/contact/approve	Approuve ou rejette le brouillon + envoi SMTP
GET	/candidats	Liste les candidats RH
POST	/candidats	Crée un candidat
PUT	/candidats/{id}	Met à jour un candidat
PATCH	/candidats/{id}/status	Change le statut d'un candidat
DELETE	/candidats/{id}	Supprime un candidat
POST	/email/check	Déclenche la lecture IMAP manuelle
GET	/email/inbox	Résumé de la boîte de réception
GET	/email/messages	50 derniers messages persistés
POST	/email/send	Envoie un courriel SMTP direct
POST	/chat/stream	SSE chat streaming avec Graph RAG

9. Infrastructure & déploiement

ManelCore est conçu pour s'exécuter entièrement en local (**local-first**). Il n'y a pas de dépendance à des services cloud : le LLM tourne sur LM Studio, la base de données sur Neo4j Desktop, et le backend sur uvicorn directement. Pas de Docker requis.

Pré-requis système

Composant	Version	Notes
Python	3.14+	Avec venv dans /Users/mazong/Documents/ManelCore/.venv
LM Studio	Dernière	Modèle chargé : google/gemma-4-e4b sur :1234
Neo4j Desktop	5.x	Base 'neo4j' sur bolt://localhost:7687
Flutter	3.x	SDK installé, cible macOS/Linux/iOS/Android
Compte email	IMAP/SMTP	iCloud, Gmail, ou domaine personnalisé supporté

Procédure de démarrage complète

1	Démarrer LM Studio Charger google/gemma-4-e4b, activer le serveur local sur :1234
2	Démarrer Neo4j Desktop Lancer la base 'neo4j', vérifier bolt://localhost:7687
3	Configurer l'environnement Copier .env.example → .env et remplir les valeurs
4	Démarrer le backend cd backend && python main.py (uvicorn sur :8000)
5	Démarrer le frontend Flutter cd manelcore && flutter run -d macos (ou linux)
6	Configurer via l'interface Page Paramètres → remplir profil entreprise et credentials

10. Flux de données end-to-end

Cycle complet de veille d'opportunités

Frontend	L'utilisateur clique 'Cycle complet' → POST /agent/run
Backend	FastAPI démarre create_explorer_graph() en tâche asyncio
Explorer	load_profile: charge profil + secteurs depuis Neo4j
Explorer	generate_queries: LLM génère 3 requêtes optimisées
Explorer	Fan-out parallèle: search_seao + search_linkedin + search_indeed
Explorer	rank_and_analyze: LLM score + analyse, MERGE dans Neo4j
Explorer	Interruption avant human_review — état suspendu en mémoire
Backend	Chaque étape pousse un événement JSON dans browser_live_queue
Frontend	GET /agent/stream consomme le SSE et affiche en temps réel
Validation	Utilisateur valide les opportunités dans la page Validations
Contact	Pour une opportunité + contact: POST /contact/draft
Explorer	Contact Agent: fetch_history → draft_response (LLM) → interrupt
Frontend	Affiche le brouillon — utilisateur approuve ou modifie
Contact	POST /contact/approve → send_email: SMTP + persistance Neo4j

11. Configuration & variables d'environnement

Variable	Défaut	Description
API_KEY	lm-studio	Clé factice pour LM Studio (interface OpenAI)
MODEL	google/gemma-4-e4b	Identifiant du modèle LLM
MODEL_BASE_URL	http://localhost:1234/v1	URL de base de l'API LLM
NEO4J_URI	bolt://localhost:7687	URI de connexion Neo4j
NEO4J_USERNAME	neo4j	Nom d'utilisateur Neo4j
NEO4J_PASSWORD	(vide)	Mot de passe Neo4j
NEO4J_DATABASE	neo4j	Nom de la base de données
MAILER_EMAIL	(vide)	Email principal (prospection)
MAILER_PASSWORD	(vide)	Mot de passe email principal
MAILER_IMAP_SERVER	mail.votredomaine.ca	Serveur IMAP principal
MAILER_HR_EMAIL	(vide)	Email RH (réponses candidatures)
MAILER_HR_PASSWORD	(vide)	Mot de passe email RH
MAILER_HR_IMAP_SERVER	mail.votredomaine.ca	Serveur IMAP RH
MAILER_COMPANY_EMAIL	(vide)	Email général entreprise
MAILER_COMPANY_PASSWORD	(vide)	Mot de passe email général
MAILER_COMPANY_IMAP_SERVER	mail.votredomaine.ca	Serveur IMAP général
TELEGRAM_BOT_TOKEN	(vide)	Token du bot Telegram (@BotFather)
SEARCH_LLM_TIMEOUT_SECONDS	30	Timeout LLM pour les recherches (secondes)
ANONYMIZED_TELEMETRY	false	Désactive la télémétrie LangChain

12. Sécurité & bonnes pratiques

Points de vigilance actuels

- **CORS ouvert** : le middleware FastAPI autorise `allow_origins=["*"]`. En production, restreindre aux origines connues.
- **Authentification** : l'API REST n'implémente pas encore d'authentification. Recommandation : ajouter JWT ou API Key en en-tête.
- **Fichier .env** : ne jamais committer le fichier .env contenant les secrets. Il est correctement dans .gitignore.
- **Mot de passe Neo4j** : stocké en clair dans .env. Utiliser un gestionnaire de secrets en production.
- **Endpoint /data/erase** : supprime toutes les données sans confirmation. Protéger par authentification et confirmation double.
- **Local-first** : en fonctionnement local, le risque d'exposition réseau est faible. Ajouter un pare-feu si déployé sur un serveur partagé.
- **Pièces jointes** : les métadonnées sont collectées mais pas le contenu binaire — évite les risques de stockage de fichiers malicieux.

Human-in-the-loop — Garantie de contrôle

ManelCore est architecturalement conçu pour maintenir le contrôle humain sur toutes les actions à impact externe :

- Aucun courriel n'est envoyé sans `approved=True` explicite dans l'état du Contact Agent.
- L'Agent Explorer s'interrompt avant `human_review` — les opportunités ne sont pas automatiquement acceptées.
- L'Email Agent peut générer des brouillons mais la validation de l'envoi reste humaine.

13. Roadmap & évolutions

Fonctionnalités en cours / à implémenter

Priorité	Fonctionnalité	Description
Haute	Authentification API	JWT ou API Key pour sécuriser tous les endpoints
Haute	Page Planificateur	Agenda intégré pour planifier les relances et échéances
Haute	Notifications desktop	Alertes macOS/Linux pour nouvelles opportunités
Moyenne	Export PDF/Excel	Export des opportunités et rapports pour présentation
Moyenne	Vector embeddings	RAG sémantique avec pgvector ou Neo4j vector index
Moyenne	Webhook SEO	Abonnement aux nouvelles publications SEO en temps réel
Basse	Multi-langue	Interface en anglais pour les opportunités fédérales
Basse	Docker Compose	Conteneurisation pour déploiement sur serveur
Basse	Tests automatisés	Suite de tests pytest pour le backend et les agents

Documentation générée automatiquement — ManelCore v1.0 — Mai 2025

Manel Canada | zebazemadric@icloud.com